

Image delivery for low-cost phones

A system-design walkthrough of the `fastimages` feed — ComposeLearning

Jetpack Compose Coil 3 OkHttp 5 India-first constraints

The thesis. Showing an image is a UI problem. Showing it fast on a ₹9,000 phone on a congested cell is a systems problem, and the answer is not "download a smaller image" — it is a *decision function* that runs on the device, resolves capability into a small set of variant parameters, and keeps every cache layer between the phone and the blob store shareable. Screen density is the input everyone reaches for first and it is the one that misleads you most.

1. The problem and the constraints

A home-screen grid of ~120 food photos. The user opens the app, sees tiles immediately, and flings. What "good" means, in priority order:

- 1. **Something is on screen in the first frame.** Never blank, never white, never jumping.
- 2. **Scroll never drops a frame**, including while images arrive and decode.
- 3. **Bytes are proportionate** to what the user can actually see and afford.
- 4. **Degradation is graceful:** slow link, cheap device, Data Saver on, network lost mid-scroll.

The constraints that make it interesting:

Constraint	Consequence
Budget phone: 1080×2400 screen, 2–3 GB RAM, 8 slow cores, ~96 MB heap	A high-density screen on a device that cannot afford high-density images. Decode CPU and bitmap memory bind before bandwidth does. One 1080×1080 ARGB_8888 bitmap is 4.4 MB of heap.
Congested cellular: 300–800 kbit/s, 200–600 ms RTT	Round trips hurt more than bytes. Anything needing a second request per tile is suspect.
Metered data, Data Saver commonly enabled	Speculative work is spending someone else's money. It needs an off switch driven by user intent.
Tens of thousands of Android SKUs	Per-device responses are unshareable. Cache keys must describe <i>bytes</i> , not devices.

2. Architecture

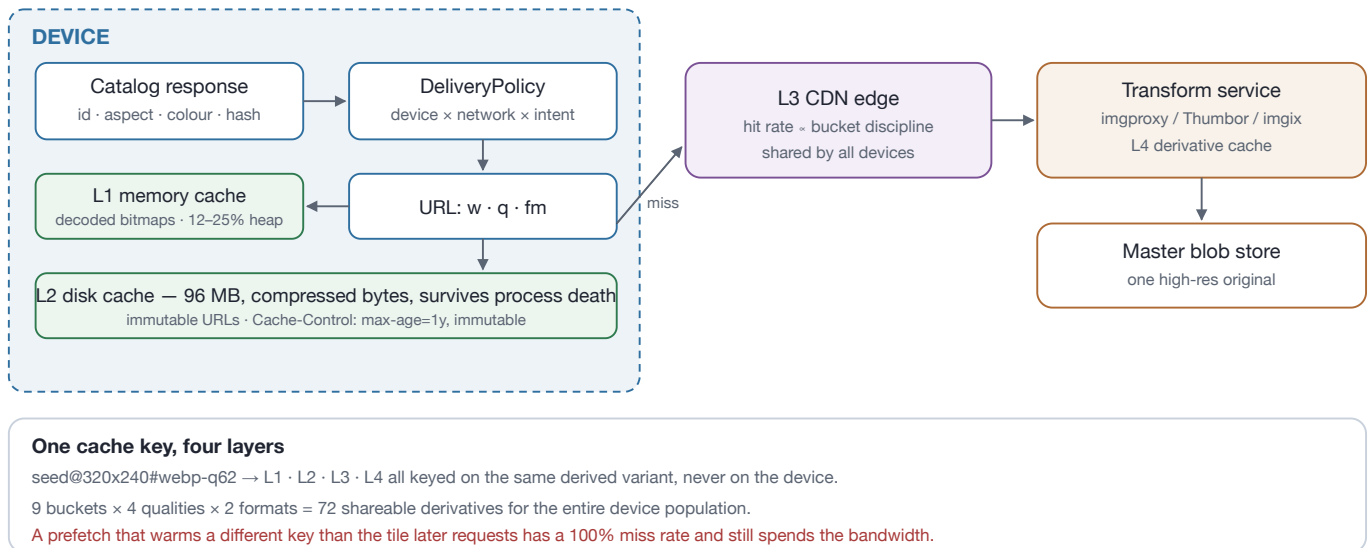


Figure 1. Four cache layers, one key. The client composes the URL; every layer downstream is shared.

The question interviewers push on: "why not send the device profile to the server and let it decide?" Because the response would then vary per device model. You would need `Vary` headers, and the CDN edge — the layer doing the most work — would fragment across tens of thousands of SKUs. Resolving locally turns that population into ~72 shared objects. Server-side device detection via User-Agent is a web-era pattern and its cost is exactly this.

3. The decision function

Everything reduces to one expression, evaluated on the device:

```
variant = f(layout size, density, device tier, network tier, user intent)
```

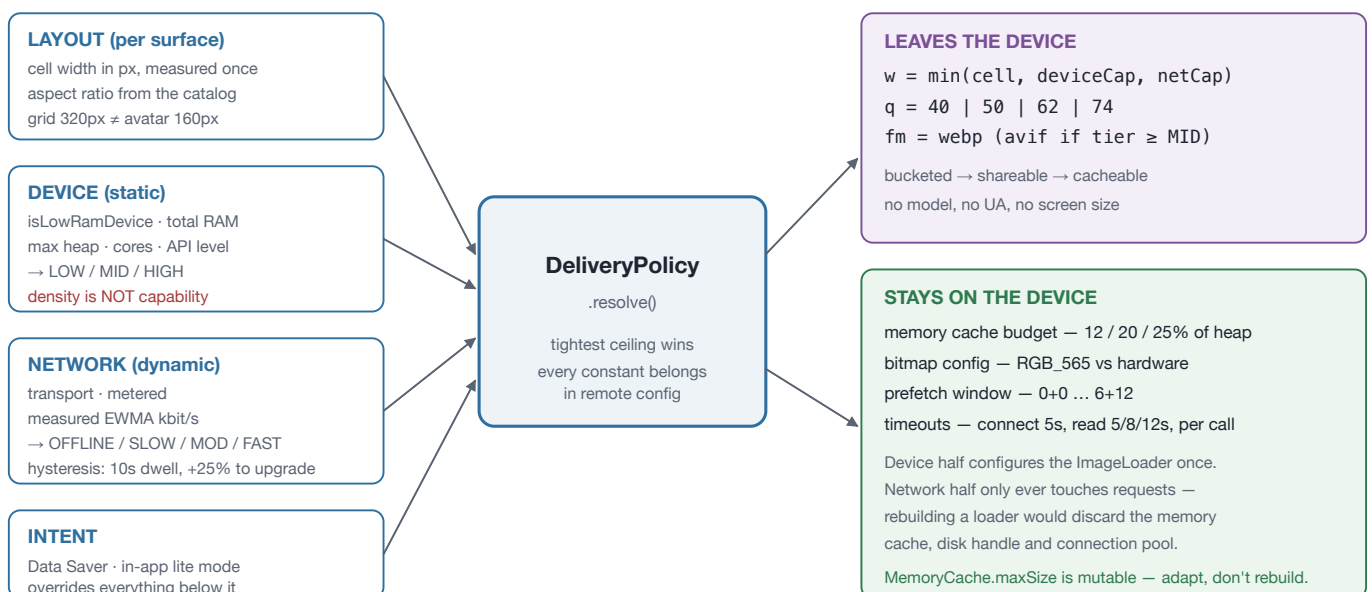


Figure 2. Capability is resolved locally. Only three parameters cross the network boundary.

Why measure throughput yourself

`NetworkCapabilities.getLinkDownstreamBandwidthKbps()` is a radio *capability hint*, not a measurement — it will report "LTE, 20 Mbps" on a congested cell delivering 300 kbit/s. So throughput comes from `okhttp3.EventListener.responseBodyStart/End`, which brackets **body bytes only**. DNS, TCP, TLS and time-to-first-byte are latency; folding them into a bandwidth figure makes a fast link to a distant server look slow.

Tier flapping is a real bug, not a theoretical one. The tier feeds the URL, so every flip changes the cache key and re-downloads the visible screen. Four defences: re-classify on a 5 s timer rather than per sample; require a 25% margin to move *up* a tier; enforce a 10 s minimum dwell; and publish a rounded kbit/s so the policy never sees jitter that only the debug panel should see.

4. One tile's request lifecycle

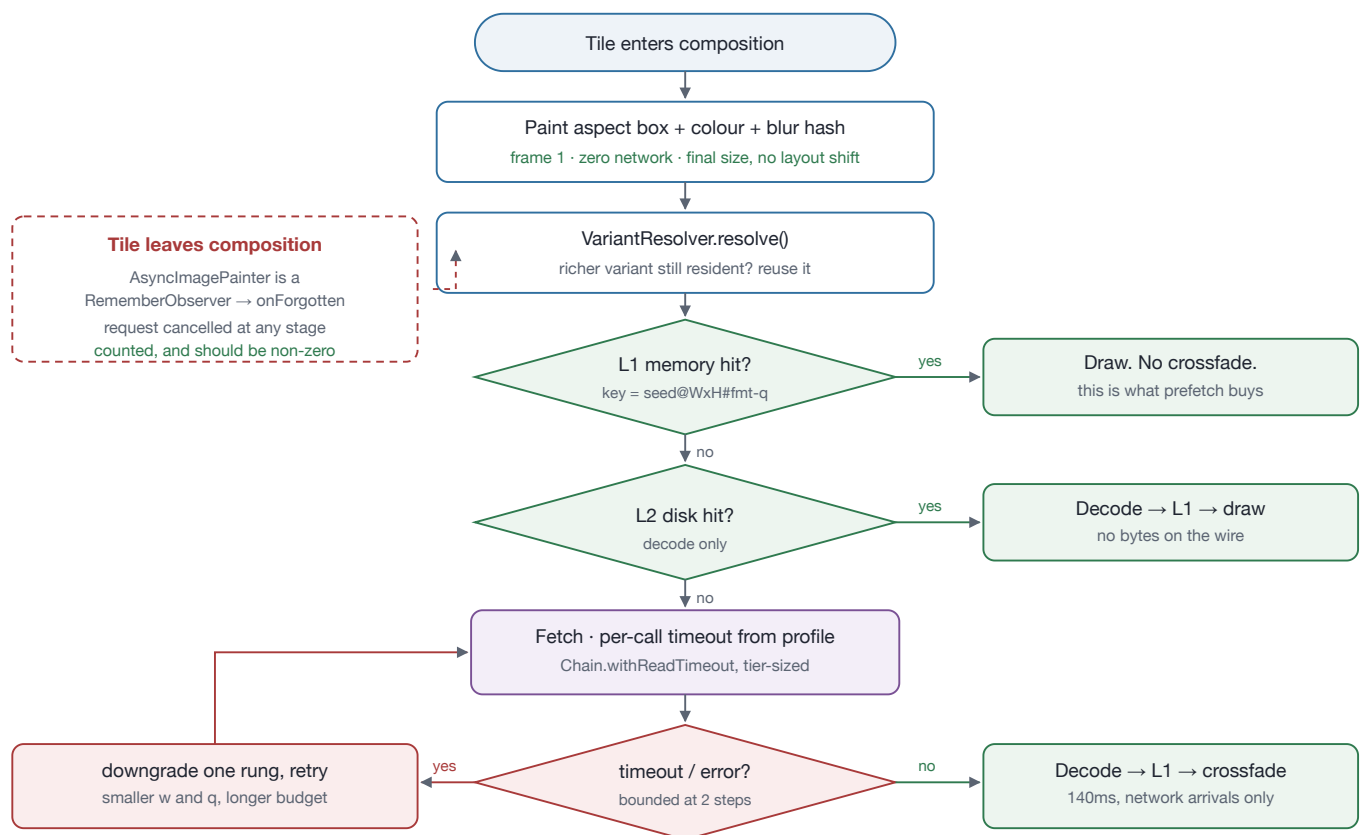


Figure 3. Three cache tiers before a byte moves; timeout downgrades rather than retries; cancellation can interrupt any stage.

Calibration beats cleverness — a war story from this very feature. The first version of this ladder used a 1.5 s read budget on a fast link, which is correct for a CDN edge returning a pre-warmed derivative in under 200 ms. The demo's backend is not that: it 302-redirects and *generates* the crop, measuring **1.25–1.68 s TTFB from a desktop on fibre under a 16-request burst**. Every request timed out, downgraded twice, hit the cap and gave up — the feed rendered as flat colour rectangles. Two things to say about it:

- A timeout ladder is derived from the **p95 TTFB of your real backend under your own concurrency**, and it is the first thing you put behind remote config: a backend regression turns an aggressive client budget into a self-inflicted outage.
- **A read timeout is not a total call budget.** "Cancel anything over 1.5 s" is `callTimeout / Call.timeout()`; an interceptor can only shape connect, read and write. And connect must be sized separately — DNS + TCP +

TLS is a fixed cost that does not scale with throughput, so folding it into the read budget lets the tighter constraint govern both.

Why downgrade instead of retry. Retrying the same variant on a link that just proved it cannot deliver it in time only spends the budget again. Asking for less is the only retry that changes the odds. The retry announces itself with a request header so the interceptor grants a longer budget — and that header is **stripped before the request leaves the device**, because if it reached the CDN it would fragment the edge cache key.

5. Prefetching, and why it is a two-tier bet

Scrolling down →

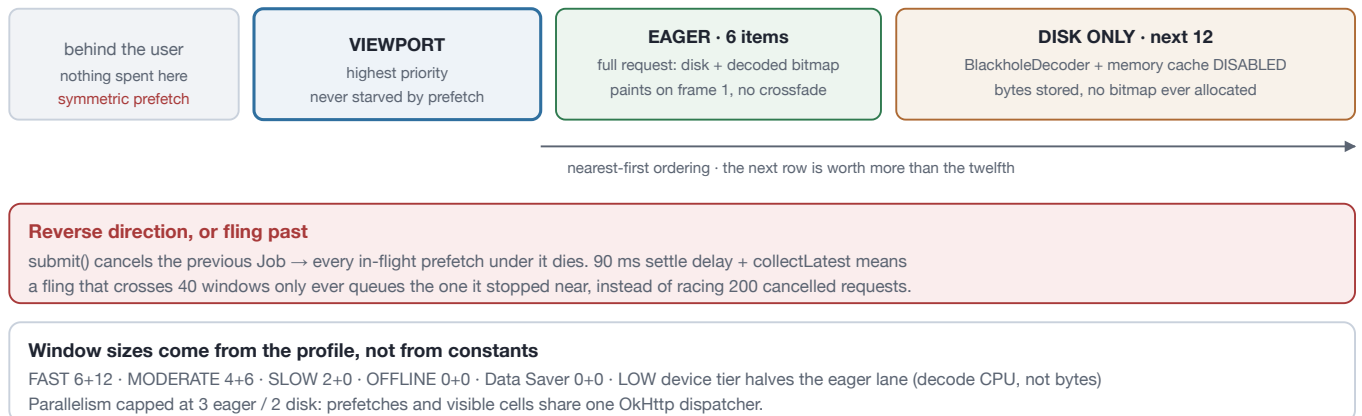


Figure 4. Direction-biased, two-tier, cancellable, and sized by policy.

6. What is done for performance

Layer	Technique	Mechanism	Why it matters
Transfer	Bucketed WebP variants	ImageCdn width ladder + q + .webp	~200 KB master → ~15 KB thumbnail for identical on-screen pixels
Transfer	Per-surface sizing	grid 320px 4:3 vs avatar 160px 1:1	Where most of the "our images are huge" problem actually lives
Transfer	maxRequestsPerHost = 12	OkHttp Dispatcher	Default is 5 per host ; a CDN is one host, so tile 6 waits on tile 1 while the radio idles
Transfer	Warm connection pool	8 idle / 5 min keep-alive	A TLS handshake costs more than a 15 KB WebP
Caching	Immutable content-addressed URLs	max-age=1y, immutable	No invalidation problem, ever
Caching	Shared cache key discipline	VariantResolver is the only producer	Prefetch and display provably agree; otherwise prefetch is pure waste

Caching	Never downgrade a decoded image	per-image richest-variant ledger + residency check	Leaving WiFi must not re-fetch the viewport at lower quality
Memory	Cache budget as a share of heap	12 / 20 / 25% by tier, <code>maxSize</code> mutable	25% of a 96 MB heap starves the rest of the app
Memory	Hardware bitmaps by default	Coil <code>allowHardware</code> , API 26+	Pixels live outside the Java heap entirely
Memory	Far-prefetch without decoding	<code>BlackholeDecoder</code> + memory cache off	Bytes cached, zero bitmaps, no eviction of near images
Render	Aspect ratio from the payload	<code>Modifier.aspectRatio(dish.aspectRatio)</code>	Arriving bitmaps never re-trigger layout for the rest of the grid
Render	Placeholder in the catalog response	dominant colour + 36-char blur hash	Something on screen in frame 1 with zero extra requests
Compose	Stable keys + <code>contentType</code>	<code>key = DishCard::id</code>	Slot reuse instead of subtree rebuild; prepends don't re-bind the world
Compose	<code>@Immutable</code> holder for the loader	<code>FeedImageEngine</code>	<code>ImageLoader</code> as a bare param makes the tile unskippable — recomposing every scroll frame
Compose	Cell width computed once	<code>LocalWindowInfo.containerSize</code>	Avoids a <code>BoxWithConstraints</code> subcomposition per tile
Compose	Scroll read only in <code>snapshotFlow</code>	inside a coroutine, never in composition	Reading <code>layoutInfo</code> in a composable body recomposes the screen every frame
Compose	Metrics polled, not pushed	atomics + 400 ms <code>produceState</code>	Hundreds of events per fling would make the measurement cause the jank

7. What is done to make scrolling smooth — the cancellation story

Smoothness is mostly about *work you refuse to do*. Four mechanisms, at four scopes:

Scope	Trigger	Mechanism	Observable as
One tile	tile scrolls out of composition	<code>AsyncImagePainter</code> is a <code>RememberObserver</code> ; <code>onForgotten</code> disposes the request	<code>cancelled</code> counter
Prefetch window	direction change / new window	<code>submit()</code> cancels the previous <code>Job</code> ; children die with it	in-flight count drops

Fling	40 windows crossed in 400 ms	90 ms settle delay + <code>collectLatest</code>	only one window is ever queued
Screen	navigating away	<code>rememberCoroutineScope</code> cancels; loader and caches survive	no orphaned work, warm return visit

The counter-intuitive metric: a healthy fling *should* produce cancellations. Zero cancellations during fast scrolling means requests are outliving their views — which is both wasted bandwidth and a decode competing with the frame you are trying to draw. The panel surfaces `cancelled` for exactly that reason.

The other half of smoothness is bounded concurrency. Prefetches and visible cells share one OkHttp dispatcher, and HTTP/2 stream priority is not something OkHttp exposes usefully — so priority is enforced structurally instead: 3 parallel eager prefetches, 2 disk-only, nearest first, and the window shrinks on slow links and low-tier devices.

8. What is done to track failures

Instrumentation lives in a Coil `EventListener` plus an OkHttp `EventListener`, writing to plain atomics that the UI polls. Nothing is pushed into snapshot state.

Signal	Source	What it tells you
bytes on the wire	OkHttp <code>responseBodyEnd(call, byteCount)</code>	Exact, unlike <code>Content-Length</code> , which is absent under chunked encoding and lies under compression. Cache hits never reach it.
memory / disk / network split	Coil <code>SuccessResult.dataSource</code>	The headline number: share of displayed images that never touched the network
fetch ms, decode ms	<code>fetchStart/End</code> , <code>decodeStart/End</code>	Separates a bandwidth problem from a CPU problem — the distinction that decides what to fix on a low-tier device
cancelled	<code>onCancel</code>	Scroll hygiene (see above)
downgrades	timeout fallback path	How often the aggressive timeout is firing — i.e. whether the budget is calibrated
failed	<code>onError</code>	Terminal failures after the ladder bottoms out
prefetch completions	<code>Extras.Key</code> tag on the request	Speculative work kept out of the display statistics — an untagged prefetch inflates your hit rate and hides the truth
bitmap cache occupancy	<code>MemoryCache.size / maxSize</code>	Whether the budget is the binding constraint

In production, every one of these is segmented by device tier × network tier. A global p50 hides the users this entire design exists for. The SLIs worth alerting on:

- image p95 latency, per segment — not a global average
- cache hit rate per layer: L1 memory, L2 disk, L3 CDN edge
- bytes per session, and *prefetch waste rate* (prefetched but never shown)
- time to first meaningful image on the home screen

- timeout rate and downgrade rate — a rising downgrade rate means the timeout ladder needs remote-config surgery, not a code change

9. Trade-offs worth raising unprompted

Choice	The tension
AVIF vs WebP	AVIF is 20–30% smaller <i>and</i> materially slower to decode on weak CPUs. On a low-tier device it can cost more in decode than it saves in transfer. "Fewer bytes" and "faster" are not the same thing. Gate it on tier and A/B test — do not enable it globally because a blog post said it is smaller.
HARDWARE vs RGB_565	Mutually exclusive. Hardware bitmaps keep pixels out of the Java heap but cost ~1 FD each and forbid pixel access; RGB_565 halves heap but is software and wrong wherever alpha exists. Many teams keep hardware everywhere and cap the <i>bucket</i> instead — the lever that cuts transfer and decode at once.
BitmapPool / inBitmap	Do not add this. Correct advice in the Glide/Fresco era; obsolete now. ART's generational GC made short-lived bitmap allocation cheap, inBitmap requires software bitmaps (so a pool forces allowHardware(false) — trading the big win for a small one), and hardware bitmaps cannot be pooled at all because their pixels live in graphics memory. Coil removed its pool deliberately.
Bucket granularity	Fewer buckets means a hotter edge cache and a smaller derivative cache, but more over-fetch per device. More buckets is the reverse. It is a cache-efficiency vs byte-efficiency dial, and it belongs in remote config.
Prefetch aggressiveness	Every prefetch is a bet placed with the user's data allowance. Track the waste rate; if it is high you are buying latency with someone else's money.
Quality-tiered URLs	Making quality part of the URL means a network change changes the cache key. That is why the never-downgrade ledger exists — without it, adaptivity actively harms the user who just walked out of WiFi.
Timeout aggressiveness	An aggressive budget converts a slow backend into a failed one, and the retry it triggers makes the congestion worse. Calibrate from measured p95 TTFB, keep it in remote config, and make failure downgrade rather than repeat. Also instrument the downgrade rate — a rising one is the early warning that the budget no longer matches the backend.
Separate LQIP request	Rejected. On a 400 ms-RTT link, a second request per tile doubles round trips before anything is visible. A hash that rides in the catalog JSON is close to free.

10. Implementation map

File	Responsibility
<code>DevicePolicy.kt</code>	Hardware signals → LOW/MID/HIGH → cache budget, bucket ceiling, bitmap config, AVIF eligibility. Static for the process.
<code>NetworkMonitor.kt</code>	Transport, metered, Data Saver, and a measured EWMA throughput → tier, with dwell time and upgrade margin to stop flapping.
<code>DeliveryPolicy.kt</code>	The decision function. <code>DeliveryProfile</code> + the downgrade ladder + the volatile handoff to the network layer.

<code>ImageCdn.kt</code>	URL and cache-key construction: width ladder, quality, format, and the naive master for comparison.
<code>VariantResolver.kt</code>	The single producer of variants. Shared by tiles and prefetcher; owns the never-downgrade ledger.
<code>FeedImageEngine.kt</code>	Tuned <code>ImageLoader</code> , <code>OkHttp</code> dispatcher/pool, transport measurement listener, adaptive per-call timeout interceptor, <code>@Immutable</code> holder.
<code>ImagePrefetcher.kt</code>	Two-tier, direction-biased, cancellable prefetch with bounded parallelism.
<code>ImageLoadMetrics.kt</code>	Pipeline instrumentation as atomics; prefetches tagged out of the display stats.
<code>MicroThumb.kt</code>	Blur-hash placeholder that ships in the catalog payload. <code>BlurHash</code> / <code>ThumbHash</code> stand-in.
<code>FeedModels.kt</code>	The catalog shape: ids and metadata, never finished URLs . Aspect ratio bucketed to two crops.
<code>FastImageFeedScreen.kt</code>	Grid, tiles, timeout-fallback state, metrics panel.
<code>DeliveryPolicyPanel.kt</code>	Makes the decision visible and forceable — the only practical way to test the low-end path.

11. Honest scorecard

Production concern	In the sample
Bucketed WebP variants, per-surface sizing	yes
Device tiering → cache budget, bucket cap, bitmap config	yes
Network tiering with measured throughput + anti-flap	yes
Data Saver / lite mode honoured	yes
Blur-hash placeholder in the payload + payload aspect ratio	yes
Two-tier direction-biased prefetch, decode-free far tier	yes
Shared cache keys; never downgrade a decoded image	yes
Adaptive per-call timeouts (connect/read split) → downgrade ladder	yes
Loader-level debug logging, so a failed load is not silent	yes
Cancel on detachment, and instrumented	yes
In-app metrics for bytes, cache split, fetch/decode split	yes
Remote config over every ladder and cut-off	no — the one thing you must not ship as constants
Telemetry segmented by device × network tier	no
Real <code>BlurHash</code> / <code>ThumbHash</code> with a server-side encoder	no — synthesised locally, since <code>picsum</code> cannot describe its own photos

Progressive JPEG	no — and <code>BitmapFactory</code> gives no incremental display for free
Paging 3, <code>PersistentList</code> , baseline profile	no

12. If you get 60 seconds to answer

"We never send the master image and we never send device info. The client measures its cell width, classifies the device from RAM and heap and the network from measured throughput, and folds those into three URL parameters — width bucket, quality, format. Buckets keep the CDN edge and our disk cache shareable across the whole device population. A dominant colour and a 30-byte blur hash ride in the catalog JSON so the first frame is never empty and the aspect ratio is known before any byte arrives. We prefetch two tiers ahead in the direction of travel — decoded for the next six, bytes-only for the twelve after that — and cancel the window on direction change. On a low-tier device the binding constraint is decode CPU and bitmap memory, not bandwidth, so the cache budget is a share of the heap and the prefetch window halves. Timeouts are per-call and adaptive; on expiry we ask for a smaller variant rather than retrying the same one. And we measure the cache split, the fetch/decode split and the cancellation rate, segmented by tier — because a global p50 hides exactly the users this design is for."

Generated from `app/src/main/java/com/example/composelearning/fastimages/` · code-level companion: `FASTIMAGES.md` in the same package.